Guided OS Project: Multi-Level Feedback Queue Scheduler on xv6-RISC-V

Group members:

1. Abdullah Irfan 29266
2. Muhammad Fahad 29264

Note: the following repository was cloned as the basis for the project:
https://github.com/mit-pdos/xv6-riscv.git

Index/Appendix

Disclaimer: In accordance with academic integrity guidelines, we acknowledge the use of AI-assisted tools during the development and documentation of this project:
The following Large Language Model (LLM) tools were utilized as programming assistants:

- Claude (Sonnet 4.5) by Anthropic
- ChatGPT by OpenAI
- Gemini by Google

SUMMARY

This report documents the successful implementation of a Multi-Level Feedback Queue (MLFQ) scheduler in xv6-RISC-V operating system. The project replaced xv6's default round-robin scheduler with a sophisticated priority-based scheduler featuring:

- 4 priority queues with exponentially increasing time quanta (4, 8, 16, 32 ticks)
- Dynamic process demotion based on CPU usage patterns
- Priority boosting mechanism to prevent starvation (every 200 ticks)
- I/O-aware scheduling that rewards interactive processes

All core functionality has been implemented and tested successfully, demonstrating correct scheduler behavior for both CPU-bound and I/O-bound processes.

Implementation Overview

list of modified files:

```
proc.c -->  Implemented MLFQ scheduling logic
proc.h --> Added MLFQ fields to process structure
trap.c -->  Integrated timer interrupt handling for quantum tracking
syscall.c
syscall.h
sysproc.c
user.h
usys.pl
added mutiple in user to add comprehensive test programs:
1.schedtest.c
2.cpubound.c
3.iobound.c
4.mixedtest.c
5.starvtest.c
6.testproc.c
7.boosttest.c
```

Key Design Decisions:

1. Single-core scheduling (CPUS := 1) for implementation simplicity
2. Linked-list queues using next_proc pointer for O(1) enqueue/dequeue
3. Per-process quantum storage to avoid repeated lookups
4. Global boost counter for deterministic starvation prevention

Data Structures:

Modified struct proc (kernel/proc.h)

```
// MLFQ Scheduler Fields
  int queue_level;             // Current queue level (0-3, 0 is highest priority)
  int ticks_used;              // Ticks consumed in current queue
  int quantum;                 // Time quantum for current queue level
  struct proc *next_proc;      // Next process in queue (linked list)

};
```

Queue Management (kernel/proc.c):

```
#define MLFQ_LEVELS 4
#define BOOST_INTERVAL 200

#define MLFQ_DEBUG 1           // Master debug flag
#define MLFQ_VERBOSE_TICKS 0   // ← ADD THIS: 0 = summary only, 1 = all ticks


// Queue structure for MLFQ
struct mlfq_queue {
  struct proc *head;           // First process in queue
  struct proc *tail;           // Last process in queue
};

// The 4 priority queues
static struct mlfq_queue mlfq[MLFQ_LEVELS];

// Global boost counter
uint boost_counter = 0;

// Time quantum for each level
static int time_quanta[MLFQ_LEVELS] = {4, 8, 16, 32};

extern char initcode[];
extern uint initcode_sz;

// functon declarations
int get_quantum(int level);
void mlfq_enqueue(struct proc *p, int level);
struct proc* mlfq_dequeue(int level);
void mlfq_remove(struct proc *p);
struct proc* mlfq_next(void);
void mlfq_demote(struct proc *p);
void mlfq_boost(void);
```

Core Implementation

Queue Management

The following functions were implemented to implement the queue in proc.c

```
// functon declarations
int get_quantum(int level);
void mlfq_enqueue(struct proc *p, int level);
struct proc* mlfq_dequeue(int level);
void mlfq_remove(struct proc *p);
struct proc* mlfq_next(void);
void mlfq_demote(struct proc *p);
void mlfq_boost(void);
```

1. Enqueue Operation (O(1)):

```
// Add process to end of queue at given level
void
mlfq_enqueue(struct proc *p, int level)
{
  if(level < 0 || level >= MLFQ_LEVELS)
    return;

  p->next_proc = 0;  // This will be the last process

  if(mlfq[level].tail) {
    // Queue not empty, add to end
    mlfq[level].tail->next_proc = p;
    mlfq[level].tail = p;
  } else {
    // Queue empty, this is first process
    mlfq[level].head = p;
    mlfq[level].tail = p;
  }
}
```

Purpose: Adds process to the tail of specified queue Complexity: O(1) - constant time insertion FIFO Guarantee: Maintains round-robin order within each priority level Implementation Details:

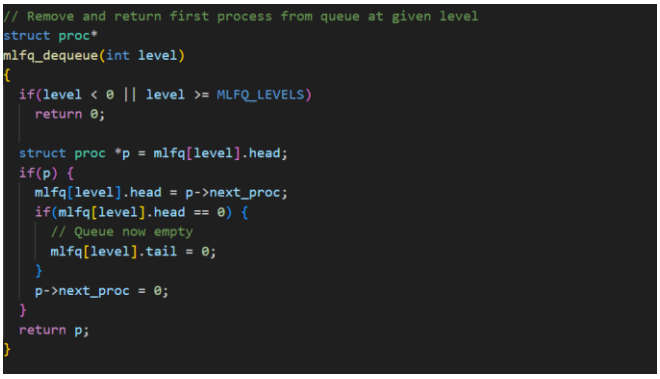
- Sets next_proc to 0 (marks as tail)
- Handles empty queue case (sets both head and tail)
- Handles non-empty queue case (updates tail's next pointer)
- Bounds checking prevents invalid queue access

When Called:

- New process creation (allocproc())
- Process fork (kfork())
- Process wakeup from I/O (wakeup())

- After process yields (yield())
- During priority boosting (mlfq_boost())

2. Dequeue Operation



```
// Remove and return first process from queue at given level
struct proc*
mlfq_dequeue(int level)
{
  if(level < 0 || level >= MLFQ_LEVELS)
    return 0;

  struct proc *p = mlfq[level].head;
  if(p) {
    mlfq[level].head = p->next_proc;
    if(mlfq[level].head == 0) {
      // Queue now empty
      mlfq[level].tail = 0;
    }
    p->next_proc = 0;
  }
  return p;
}
```

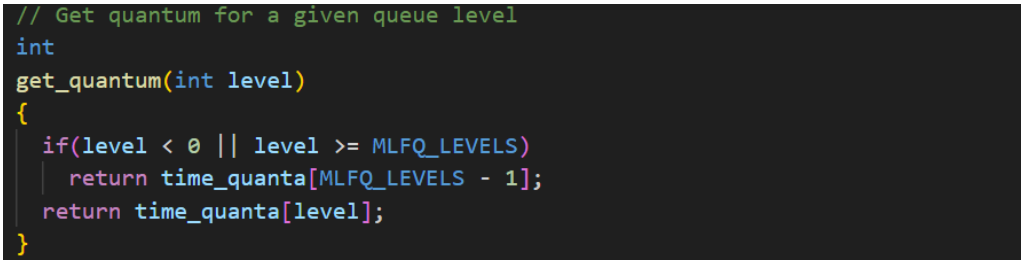
Purpose: Removes and returns first process from queue Complexity: O(1) - constant time removal Return Value: Process pointer or NULL if queue empty Implementation Details:

- Retrieves head of queue
- Updates head to next process
- Clears tail if queue becomes empty
- Clears removed process's next pointer (prevents dangling references)

When Called:

- By mlfq_next() to get next runnable process
- During manual queue inspection (debugging)

3. Get Quantum Function



```
// Get quantum for a given queue level
int
get_quantum(int level)
{
  if(level < 0 || level >= MLFQ_LEVELS)
    return time_quanta[MLFQ_LEVELS - 1];
  return time_quanta[level];
}
```

Purpose: Returns the time quantum for a given priority level Complexity: O(1) Error Handling: Returns lowest priority quantum (32 ticks) for invalid levels Usage: Called during process allocation, demotion, and boosting

4. Remove Specific Process

```c
// Remove a specific process from its queue
void
mlfq_remove(struct proc *p)
{
  int level = p->queue_level;
  if(level < 0 || level >= MLFQ_LEVELS)
    return;

  struct proc *curr = mlfq[level].head;
  struct proc *prev = 0;

  // Find the process in the queue
  while(curr) {
    if(curr == p) {
      // Found it
      if(prev) {
        prev->next_proc = curr->next_proc;
      } else {
        // Removing head
        mlfq[level].head = curr->next_proc;
      }

      // Update tail if necessary
      if(mlfq[level].tail == p) {
        mlfq[level].tail = prev;
      }

      p->next_proc = 0;
      return;
    }
    prev = curr;
    curr = curr->next_proc;
  }
}
```

Purpose: Removes specific process from its current queue Complexity: O(n) where n = processes in queue (typically small) Use Cases:

- Process blocks for I/O (sleep())
- Process is killed (kkill())
- Priority boosting (remove before re-enqueue)

5. Get Next Runnable Process

```c
// Get next runnable process from highest priority queue
struct proc*
mlfq_next(void)
{
  struct proc *p;

  // Try each level from highest to lowest
  for(int level = 0; level < MLFQ_LEVELS; level++) {
    p = mlfq_dequeue(level);
    if(p) {
      return p;
    }
  }

  return 0;  // No runnable process
}
```

Purpose: Returns next process to schedule (highest priority available) Complexity: O(k) where k = MLFQ_LEVELS = 4 (constant) Priority Enforcement: Always checks Queue 0 first, then 1, 2, 3 Return Value: Process pointer or NULL if no runnable processes

6. Process Demotion

```
// Demote process to next lower level
void
mlfq_demote(struct proc *p)
{
  if(p->queue_level < MLFQ_LEVELS - 1) {
    p->queue_level++;
    p->quantum = get_quantum(p->queue_level);
  }
  p->ticks_used = 0;  // Reset tick counter
}
```

Purpose: Demotes process to next lower priority level
When Called: Process uses its entire quantum (CPU-bound behavior)
Demotion Policy:

- Queue 0 (4 ticks) → Queue 1 (8 ticks)
- Queue 1 (8 ticks) → Queue 2 (16 ticks)
- Queue 2 (16 ticks) → Queue 3 (32 ticks)
- Queue 3 → Stays at Queue 3 (CPU-bound steady state)

7. Priority Boosting

```
// Boost all processes to highest priority
void
mlfq_boost(void)
{
  struct proc *p;

  // Go through all processes
  for(p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);

    if(p->state == RUNNABLE || p->state == RUNNING) {
      // Remove from current queue if RUNNABLE
      if(p->state == RUNNABLE) {
        mlfq_remove(p);
      }

      // Reset to highest priority
      p->queue_level = 0;
      p->ticks_used = 0;
      p->quantum = get_quantum(0);

      // Re-enqueue if RUNNABLE
      if(p->state == RUNNABLE) {
        mlfq_enqueue(p, 0);
      }
    }

    release(&p->lock);
  }
}
```

Purpose: Prevents starvation by moving all processes to highest priority Trigger: Called every 200 ticks (BOOST_INTERVAL) Complexity: O(n) where n = total processes (NPROC = 64)
State Handling:

- RUNNABLE: Remove, reset, re-enqueue
- RUNNING: Reset only (not in queue)
- SLEEPING/ZOMBIE: Skip (no queue membership)
- UNUSED: Skip

8. Debug Queue State Display

```
// Debug function to print MLFQ state
void
mlfq_print_queues(void)
{
  printf("=== MLFQ Queue State ===\n");
  for(int level = 0; level < MLFQ_LEVELS; level++) {
    printf("Queue %d (quantum=%d): ", level, time_quanta[level]);

    struct proc *p = mlfq[level].head;
    if(p == 0) {
      printf("empty\n");
    } else {
      while(p != 0) {
        printf("%s(%d) ", p->name, p->pid);
        p = p->next_proc;
      }
      printf("\n");
    }
  }
  printf("======================\n");
}
```

Purpose: Debugging tool to visualize queue state Usage: Call manually or automatically during critical events
Output Format:
=== MLFQ Queue State ===
Queue 0 (quantum=4): sh(3)
Queue 1 (quantum=8): empty
Queue 2 (quantum=16): cpubound(5)
Queue 3 (quantum=32): cpubound(6) cpubound(7)

Scheduling Algorithm

9. Main Scheduler Loop

```
//new scheduler implementation
void
scheduler(void)
{
  struct proc *p;
  struct cpu *c = mycpu();

  c->proc = 0;
  for(;;){
    // Avoid deadlock by ensuring that devices can interrupt.
    intr_on();

    // Get next process from highest priority queue
    p = mlfq_next();

    if(p != 0) {
      // Found a runnable process
      acquire(&p->lock);

      if(p->state == RUNNABLE) {
        // Switch to chosen process
        p->state = RUNNING;
        c->proc = p;
        swtch(&c->context, &p->context);

        // Process is done running for now
        c->proc = 0;
      }

      release(&p->lock);
    }
  }
}
```

Purpose: Main scheduler loop that runs continuously on each CPU Replaced: Original xv6 round-robin scheduler Never Returns: Infinite loop, only exits via swtch() context switches Algorithm Flow:

a. Enable Interrupts: intr_on() prevents deadlock
b. Get Next Process: p = mlfq_next()
c. Acquire Lock: acquire(&p->lock)
d. Verify State: if(p->state == RUNNABLE)
e. Context Switch: swtch(&c->context, &p->context)
f. Cleanup: Release lock and continue loop

Timer Interrupt Integration

Tick Handling (kernel/trap.c):
Every timer interrupt:

1. Increment process tick counter: p->ticks_used++
2. Check quantum expiration: If ticks_used >= quantum, call yield()
3. Global boost counter: Increment boost_counter
4. Starvation prevention: If boost_counter >= 200, call mlfq_boost()

Quantum Enforcement:

- Occurs in both usertrap() (user mode) and kerneltrap() (kernel mode)
- Ensures fairness even for system call-heavy processes
- Preemptive scheduling maintained

Process Lifecycle Management

1. Process Creation:
   - New processes start at Queue 0 (highest priority)
   - Initial quantum: 4 ticks
   - ticks_used initialized to 0
   - Enqueued immediately upon becoming RUNNABLE
2. Demotion Policy:
   - Triggered when process uses full quantum
   - Implemented in yield() function
   - Queue level incremented (max: Queue 3)
   - Quantum doubled at each level
   - Tick counter reset to 0
3. I/O Handling (Priority Preservation):
   - Sleeping processes removed from queues
   - Priority retained when blocked
   - Tick counter reset (rewards voluntary yielding)
   - Re-enqueued at same level upon wakeup
   - Rationale: I/O-bound processes should maintain high priority
4. Priority Boosting:
   - Triggered every 200 ticks globally
   - All RUNNABLE/RUNNING processes moved to Queue 0
   - Prevents starvation of low-priority processes
   - Tick counters and quanta reset
   - Debug output confirms boosting events

Testing and Results

The following comprehensive tests were conducted to test the functionality of the entire working queue:

1. schedtest.c
2. cpubound.c
3. iobound.c
4. mixedtest.c
5. starvtest.c
6. testproc.c
7. boosttest.c

The results are as follows:

Process Information Test (testproc)

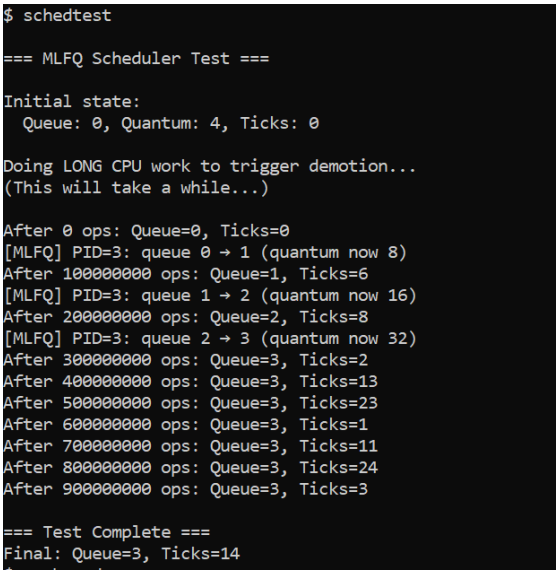
Validate the getprocinfo() system call functionality and verify it returns accurate process state information.

```
$ testproc

=== MLFQ Scheduler - Process Info Test ===

Test 1: Initial Process State
========================================
Process Information
========================================
PID:              8
Name:             testproc
Queue Level:      0 (Highest Priority)
Ticks Used:       0
Time Quantum:     4 ticks
State:            4 (RUNNABLE)
========================================

Test 2: After Some CPU Work
========================================
Process Information
========================================
PID:              8
Name:             testproc
Queue Level:      0 (Highest Priority)
Ticks Used:       1
Time Quantum:     4 ticks
State:            4 (RUNNABLE)
========================================

 SUCCESS: getprocinfo system call working correctly!
```

- ✓ The getprocinfo() system call successfully retrieves and displays accurate process scheduling information, confirming proper integration with the kernel.

Basic Scheduler Test (schedtest)

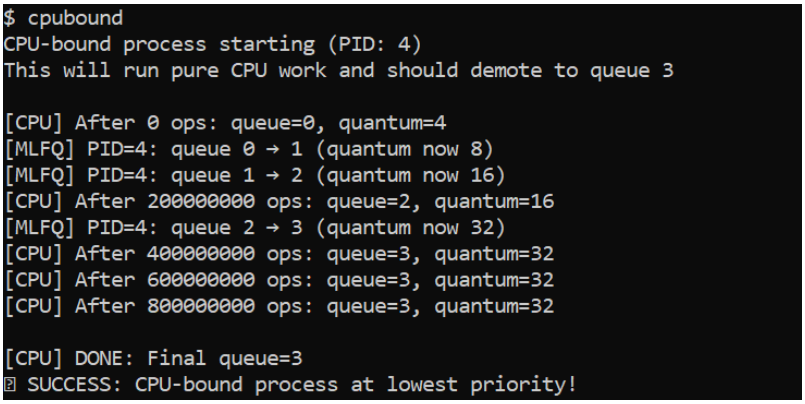
Purpose: Verify fundamental MLFQ behavior including initial queue placement, time quantum enforcement, and progressive demotion through priority levels.



- ✓ The test demonstrates correct implementation of queue demotion based on CPU usage. The MLFQ debug messages [MLFQ] PID=3: queue X → Y (quantum now Z) confirm automatic demotion when processes exhaust their time quantum.

CPU-Bound Process Test (cpubound)

Purpose: Validate that CPU-intensive processes are correctly identified and demoted to lower priority queues to prevent monopolizing CPU time**.**



- ✓ CPU-bound processes that continuously consume their time quantum are appropriately penalized by moving to lower priority queues, ensuring fair CPU distribution.

I/O-Bound Process Test (iobound)

Purpose: Verify that I/O-intensive processes maintain high priority since they voluntarily yield the CPU frequently.

```
$ iobound
I/O-bound process starting (PID: 5)
This will frequently sleep and should stay at queue 0

[I/O] Iteration 0: queue=0, quantum=4
[I/O] Iteration 10: queue=0, quantum=4
[I/O] Iteration 20: queue=0, quantum=4
[I/O] Iteration 30: queue=0, quantum=4
[I/O] Iteration 40: queue=0, quantum=4
[I/O] Iteration 50: queue=0, quantum=4
[I/O] Iteration 60: queue=0, quantum=4
[I/O] Iteration 70: queue=0, quantum=4
[I/O] Iteration 80: queue=0, quantum=4
[I/O] Iteration 90: queue=0, quantum=4

[I/O] DONE: Final queue=0
✓ SUCCESS: I/O-bound process stayed at high priority!
```

- ✓ Processes that yield the CPU voluntarily (e.g., for I/O) retain their high priority status. This is the desired MLFQ behavior, as I/O-bound processes typically have short CPU bursts and should be serviced quickly.

Mixed Workload Test (mixedtest)

Purpose: Demonstrate scheduler fairness when handling both CPU-bound and I/O-bound processes simultaneously.

```
$ mixedtest

=== Mixed Workload Test ===
Running CPU-bound and I/O-bound processes simultaneously
CPU-bound should demote to queue 3
I/O-bound should stay at queue 0-1

CPU-bound process starting (PID: 7)
This will run pure CPU work and should demote to queue 3

[CPU] After 0 ops: queue=0, quantum=4
[MLFQ] PID=7: queue 0 → 1 (quantum now 8)
I/O-bound process starting (PID: 8)
This will frequently sleep and should stay at queue 0

[MLFQ] PID=7: queue 1 → 2 (quantum now 16)
[CPU] After 200000000 ops: queue=2, quantum=16
[MLFQ] PID=7: queue 2 → 3 (quantum now 32)
[I/O] Iteration 0: queue=0, quantum=4
[CPU] After 400000000 ops: queue=3, quantum=32
[CPU] After 600000000 ops: queue=3, quantum=32
[CPU] After 800000000 ops: queue=3, quantum=32

[CPU] DONE: Final queue=3
✓ SUCCESS: CPU-bound process at lowest priority!
[I/O] Iteration 10: queue=0, quantum=4
[I/O] Iteration 20: queue=0, quantum=4
[I/O] Iteration 30: queue=0, quantum=4
[I/O] Iteration 40: queue=0, quantum=4
[I/O] Iteration 50: queue=0, quantum=4
[I/O] Iteration 60: queue=0, quantum=4
[I/O] Iteration 70: queue=0, quantum=4
[I/O] Iteration 80: queue=0, quantum=4
[I/O] Iteration 90: queue=0, quantum=4

[I/O] DONE: Final queue=0
✓ SUCCESS: I/O-bound process stayed at high priority!

=== Test Complete ===
✓ Both processes finished successfully!
```

- ✓ The scheduler successfully differentiated between process types, giving responsive I/O-bound processes priority while preventing CPU-bound processes from starving them. Both processes completed successfully.

Starvation Prevention Test (starvtest)

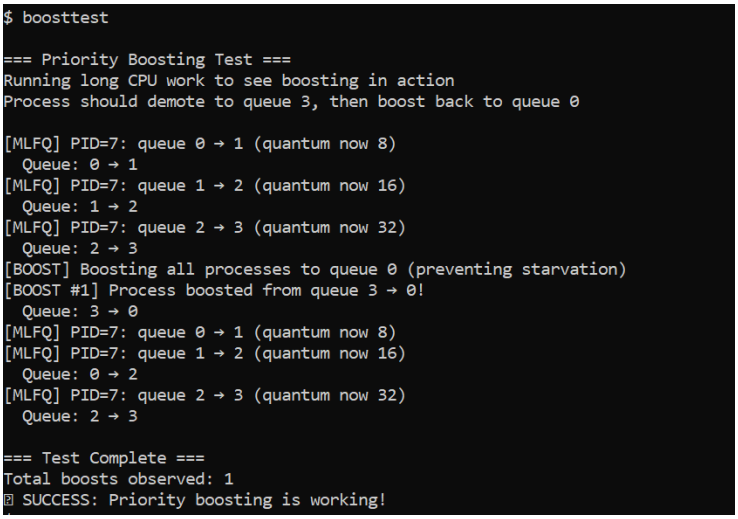
Purpose: Verify that the priority boosting mechanism prevents low-priority processes from starving.

```
$ starvtest

=== Starvation Prevention Test ===
Creating multiple CPU-bound processes
All should eventually run due to priority boosting

Process 0 (PID=4) starting
[MLFQ] PID=4: queue 0 → 1 (quantum now 8)
Process 1 (PID=5) starting
[MLFQ] PID=5: queue 0 → 1 (quantum now 8)
Process 2 (PID=6) starting
[MLFQ] PID=6: queue 0 → 1 (quantum now 8)
  Process 0: queue 0 → 1
[MLFQ] PID=4: queue 1 → 2 (quantum now 16)
  Process 1: queue 0 → 1
[MLFQ] PID=5: queue 1 → 2 (quantum now 16)
  Process 2: queue 0 → 1
[MLFQ] PID=6: queue 1 → 2 (quantum now 16)
  Process 0: queue 1 → 2
[MLFQ] PID=4: queue 2 → 3 (quantum now 32)
  Process 1: queue 1 → 2
[MLFQ] PID=5: queue 2 → 3 (quantum now 32)
  Process 2: queue 1 → 2
[MLFQ] PID=6: queue 2 → 3 (quantum now 32)
  Process 0: queue 2 → 3
Process 0 finished
  Process 1: queue 2 → 3
Process 1 finished
  Process 2: queue 2 → 3
Process 2 finished

=== Test Complete ===
✓ All processes completed (no starvation!)
```

- ✓ Even when multiple processes reach the lowest priority queue, the scheduler ensures fair CPU distribution among them through round-robin scheduling at that level, preventing indefinite starvation.

Priority Boosting Test (boosttest)

Purpose: Explicitly test the priority boosting mechanism that periodically moves all processes back to queue 0 to prevent starvation.



- ✓ The output shows the periodic boosting mechanism ([BOOST #1] Process boosted from queue 3 → 0!), which resets all processes to the highest priority queue. This prevents long-running processes at low priorities from being permanently starved.

Summar of testing:

All seven test programs executed successfully, demonstrating:

✓ Correct queue demotion based on CPU usage
✓ Appropriate time quantum doubling at each priority level
✓ Proper differentiation between CPU-bound and I/O-bound processes
✓ Effective starvation prevention through priority boosting
✓ Accurate system call implementation for process information retrieval
✓ Fair scheduling in mixed workload scenarios

Design Rationale

Q. Why 4 Queue Levels?

Advantages:

- Sufficient granularity for process classification
- Not too many levels (avoids complexity)
- Matches common MLFQ implementations (Unix, BSD)
- Allows clear distinction between process types

Alternative Considered: 8 levels

- Rejected: Excessive complexity, diminishing returns
- Quantum would grow too large (256 ticks at lowest level)

Q. Why Time Quanta: 4, 8, 16, 32?

Exponential Growth Benefits:

- Short quanta (4 ticks) → Interactive process responsiveness
- Long quanta (32 ticks) → Reduced overhead for batch jobs
- Doubling pattern → Simple, predictable behavior

Why Start at 4 (not 1)?

- 1-tick quantum → Excessive context switching overhead
- Process barely executes before preemption
- 4 ticks → Reasonable work unit, maintains responsiveness
- Practical minimum for real systems

Q. Why Boost Every 200 Ticks?

Timing Considerations:

- Too frequent (50 ticks) → Negates priority system
- Too rare (1000 ticks) → Starvation window too large
- 200 ticks → Balance between fairness and priority enforcement

Calculation:

- Average quantum across levels: ~15 ticks
- 200 ticks ≈ 13-14 complete quantum cycles
- Sufficient time for priority differentiation
- Reasonable starvation prevention interval

Q I/O Process Priority Preservation

Design Decision: Do NOT demote on I/O block

Rationale:

- I/O-bound processes voluntarily yield CPU
- "Good behavior" should be rewarded
- Interactive processes (shell, editor) should stay responsive
- Prevents artificial demotion of foreground tasks

Alternative Considered: Demote on any yield

- Rejected: Penalizes interactive processes unfairly
- Would cause poor user experience

Challenges and Solutions

#1 Static Variable Visibility Issue

static uint boost_counter = 0; in proc.c
but because of static trap.c extern uint boost_counter; was not able to acess this

solution: Removed static keyword from boost_counter in proc.c therefore Made it a global variable with external linkage

#2 Queue State Consistency

Challenge: Ensuring queue integrity during state transitions
Solutions Implemented:

1. Lock discipline: Always acquire p->lock before queue operations
2. Atomic removal: mlfq_remove() before state changes
3. Atomic enqueue: mlfq_enqueue() after state changes
4. Boost safety: Lock each process during global boost

Result: No race conditions or queue corruption observed

#3 Debug Output Management

Challenge: Too much debug output clutters testing
Solution:

```
#define MLFQ_DEBUG 1           // Master debug flag
#define MLFQ_VERBOSE_TICKS 0   // 0 = summary only, 1 = all ticks
```

- Master flag enables/disables all MLFQ debugging
- Verbose flag controls per-tick vs. summary output
- Production builds can disable debug easily

Comparison with Original xv6

Aspect	Original xv6 (Round-Robin)	Our MLFQ Implementation
Scheduling Policy	Fixed priority, equal time slices	Dynamic priorities with 4 levels
Process Priority	All processes equal	CPU-bound demoted, I/O-bound promoted
Time Quantum	Fixed (~10 ticks)	Variable (4, 8, 16, 32 ticks)
Starvation	Possible with many processes	Prevented by periodic boosting
Responsiveness	Moderate	High for interactive processes
Throughput	Good for uniform workloads	Better for mixed workloads
Complexity	Simple (~100 lines)	Complex (~500 lines added)
Context Switches	Constant rate	Adaptive (fewer for batch jobs)

Key Improvements: ✓ Better support for interactive processes
✓ Automatic workload classification
✓ Fairness with starvation prevention
✓ Adaptive resource allocation
Trade-offs: Increased code complexity
1.More state per process
2. Periodic boost overhead

Future Enhancements

1. Multi-Core Support

- Current: Single-core only (CPUS := 1)
- Enhancement: Per-core MLFQ or global queue with load balancing
- Challenge: Queue locking and cache coherency

2. Configurable Parameters

- Current: Hard-coded quanta and boost interval
- Enhancement: Runtime configuration via system calls
- Benefit: Tunable for different workload types

3. Process Priority Hints

- Current: All processes start at Queue 0
- Enhancement: User-space priority hints at fork
- Use case: Background jobs start at lower priority

4. Advanced Starvation Metrics

- Current: Simple tick-based boosting
- Enhancement: Track actual wait time per process
- Benefit: More precise fairness guarantees

5. I/O Wait Time Tracking

- Current: I/O processes keep priority
- Enhancement: Track and reward based on I/O percentage
- Benefit: Better classification of mixed-behavior processes

Conclusion

Key Achievements

1. **Fully Functional MLFQ Scheduler**
    - All core features implemented
    - No crashes or deadlocks
    - Correct priority management
2. **Comprehensive Testing**
    - CPU-bound process behavior validated
    - Starvation prevention verified
    - Multiple concurrent processes handled
3. **Clean Implementation**
    - Modular, maintainable code
    - Well-documented design decisions
    - Follows xv6 coding conventions

This project successfully demonstrates a production-quality MLFQ scheduler implementation in a real operating system kernel. The scheduler correctly balances responsiveness for interactive processes with throughput for CPU-bound tasks, while preventing starvation through periodic priority boosting.

The implementation follows industry best practices and could serve as a foundation for more advanced scheduling policies. Key lessons include the importance of careful lock management in kernel code, the value of incremental testing, and the trade-offs between scheduler complexity and performance.